CONTENTS

Evaluation Set & Scorecard — Retail / E-commerce

Author: Rishi — Intern 3 (Retail) **Purpose:** 25–30 hand-written questions with expected behaviour, + the scorecard the harness produces. Baseline-gate deliverable — exists **before** the chatbot does.

> **[TODO]** = fill once the real corpus + dataset exist (expected doc titles, real customer IDs). Questions are written now; expected sources are pinned when the corpus is built.

A. Corpus plan (phase 1) — source your own, 15–40 docs

- **Majority real public documentation.** Retailers publish extensive help centres, returns/refund/warranty/delivery terms. Public pages only. Sources listed in `data/sources.md`.
- **Four companies — Amazon, Best Buy, IKEA, Target** — chosen for differing terminology for the same thing + genuinely conflicting policies, the mess the system must handle. Source URLs in `data/sources.md`.
- **Deliberately contradicting pair: Amazon ~30-day standard returns vs Best Buy 15-day (14-day cellular; $45 restocking fee on activatable devices).** Bonus spread: IKEA 365-day, Target 90-day. The system must surface the conflict, not blend them. All 21 corpus docs are **real public pages** (no LLM-generated filler); their natural inconsistency in terms and coverage is the mess the system must handle.
- **Mix long and short docs** — uniform length hides chunking problems.

B. Dataset plan (phase 2) — generate it, synthetic

- ~4–6 SQLite tables: `orders`, `line_items`, `shipments`, `returns`, `customers`. A few hundred to a few thousand rows.

- **Consistent with the documents** — if a policy doc says refunds take 5 working days, the data shows ~5, with a few exceptions (exceptions are what agents call about).
- **Awkward cases included:** a duplicate charge, a partially shipped order, a return already under review, a customer with two accounts.
- **3–4 reference customers** documented with their IDs for demos. **[TODO: pin IDs once generated.]**
- Small enough to inspect by hand — I must know the right answer to score my own eval.

C. Questions

Phrased the way an agent asks mid-call. Each records what *should* happen. `expected_source` is **[TODO]** until the corpus is built.

DOCUMENT (7) — INCL. TWO-DOCUMENT ANSWERS + VOCABULARY DIFFERING FROM SOURCE

#	Question (agent voice)	Should do	Expected
1	”how long till they get their money back on a return?”	search	refund-window passage; note “money back” must match a “refund” chunk (vocab differs)
2	”customer’s parcel never showed up — what’s our process?”	search	delivery-failure passage
3	”can they send back opened electronics?”	search	returns-eligibility passage
4	”what’s covered under warranty and for how long?”	search	warranty passage
5	”they want to return after 40 days, are we allowed?”	search → **refuse/deny**	return-window passage; answer = no
6	”returns window AND who pays return shipping?”	search (**two documents**)	returns-policy + shipping-cost docs
7	”does every store charge a ‘restocking fee’ on returns?”	search (**vocab conflict across docs**)	Best Buy $45 activatable-device fee vs IKEA / Target (no such fee) — flag they differ

RECORD (6) — EXACT LOOKUPS, FILTERED LISTS, ≥1 AGGREGATE

#	Question	Should do	Expected
8	”status of order [REF]?”	query_orders	that order’s status
9	”did order [REF] actually ship?”	query_shipments	shipment row
10	”list this customer’s open returns”	query_returns (filtered)	filtered list
11	”how many orders has [customer] placed this year?”	query (**aggregate/count**)	correct count
12	”**how much has this customer been refunded this year?**”	query (**aggregate/sum**)	correct total
13	”which line items in order [REF] were delivered?”	query_line_items (filtered)	filtered rows

COMPOSITE (4) — NEED DOCUMENTS AND RECORDS IN ONE ANSWER

#	Question	Should do	Expected
14	”**I was charged twice — is that allowed and did it actually happen?**”	search (payments policy) + query (duplicate charge)	policy + the duplicate-charge row
15	”can they return order [REF] — what’s the window and is it eligible?”	query_orders + search (returns policy)	order date + policy → eligible/not
16	”parcel split into two — is partial delivery covered, and what shipped?”	search (delivery policy) + query_shipments	policy + partial-shipment rows
17	”refund on [REF] — how long should it take and did it go through?”	search + query_returns	policy window + return status

CROSS-SERVER (4) — ≥2 NEED ANOTHER INDUSTRY, ≥1 COMPARATIVE ACROSS TWO AT ONCE

#	Question	Should do	Expected
18	”does the **bank** show a refund for this charge yet?”	route → **Banking** server	banking record; not retail
19	”customer disputing a **telecom** bill, not our order”	route → **Telecom** server	telecom; retail refuses its part
20	”**do refund timelines differ between us and the bank?**”	**comparative — retail + Banking at once**	both policies, compared
21	”compare our return window with the **hotel’s** cancellation window”	**comparative — retail + Hospitality**	both, compared

ACTION (4) — INCL. ONE MISSING-FIELD → CLIENT MUST ASK

#	Question	Should do	Expected
22	”open a return for order [REF], item [ITEM], reason damaged”	`kb_retail_create_return`, **confirm first**	confirmation showing fields, then RMA ref
23	”start a return for this customer” (**missing order/item**)	**client asks for the missing fields**, does not invent	prompts for order_id + line_item_id
24	”raise the return we just discussed” (follow-up, multi-turn)	`kb_retail_create_return` using prior turn’s order, confirm	RMA ref
25	”open a return on order [REF]” but item already returned	`kb_retail_create_return` → error (`retryable: false`)	loud fail, no silent default

UNANSWERABLE (3) — RETRIEVAL SHOULD FAIL, SYSTEM SHOULD REFUSE

#	Question	Should do	Expected
26	”what’s the CEO’s mobile number?”	**refuse**	not in data
27	”will this product be cheaper next month?”	**refuse**	can’t predict
28	”status of order 99999999?” (**no such order**)	query → empty → **refuse honestly**	zero rows, not invented

(28 questions — inside 25–30.)

D. Scorecard — reported TWICE (baseline when retrieval works, final after tuning)

Each layer measured separately.

Retrieval (phase 1): Recall@5 ≥85% · Recall@1 ≥60% *(may argue accept-set on Recall@1 if my corpus has genuine policy conflicts — see design doc §7)*

Routing: correct server ≥90% · correct tool-type ≥90% · spurious calls ≤1/query avg · cross-server synthesis ≥80% · composite handling ≥80%

Structured (phase 2): query correctness ≥90% · numerical accuracy 100% · empty-result honesty 100%

Answer quality: groundedness 100% · citation accuracy 100% · correct refusal 100% · false refusal ≤10%

Action safety (phase 3): spurious writes 0 · fabricated fields 0 · correct action routing 100% · confirmation shown 100%

Latency: e2e p50 ≤4s · p95 ≤10s · retrieval ≤300ms · query ≤100ms · MCP overhead ≤100ms · *(warm vs cold reported separately — Ollama unloads idle models)*

Token efficiency: tokens/query reported → reduced ≥40% vs naive raw-JSON injection · no groundedness/recall regression

Robustness (pass/fail, none may crash): 1 server down · all servers down · empty query · 5000-word query · wrong-language query · off-topic question · 10000-row match · missing customer ref · two identical concurrent requests

---

E. Measurement harness (`eval/run_evals.py`)

One script, one command, prints the scorecard as one table.

- Reads this eval set (question + expected).
- Runs **each question in a fresh session** (else a question passes only because an earlier turn retrieved the right passage → numbers stop being repeatable).
- Scores each layer separately; logs per query which servers/tools were called, what they returned, latency per stage.
- **Instruments token counts from the baseline run onward** — a reduction figure with no starting point is not a measurement.
- Runs the token test **both ways** (verbose vs compact) and reports both numbers.

(Harness code lands at the baseline gate, with the first scorecard.)